

Gestion de Projet Agile

Scrum & Kanban — Rôles Product Owner / Scrum Master

Formateur : Haithem Mihoubi

Module : 1 / 3

Durée : 3 jours (21 heures)

Niveau : Mixte (débutant → intermédiaire)

Public : chefs de projet, développeurs, PO, futurs SM

Format : support étudiant — théorie, ateliers, exercices

Sommaire

Avant de commencer	3
Jour 1 — Fondamentaux de l'Agilité	4
1.1 Pourquoi l'Agilité ? Le contexte historique	4
1.2 Le Manifeste Agile : 4 valeurs, 12 principes	5
1.3 Prédicatif (cycle en V) vs Adaptatif (itératif)	6
1.4 Concepts clés	7
1.5 Atelier 1 — Diagnostic et transition vers l'Agile	8
Quiz — Jour 1	9
Jour 2 — Scrum : cadre, rôles et cérémonies	10
2.1 Le cadre Scrum en un coup d'œil	10
2.2 Les rôles (« responsabilités »)	10
2.3 Les artefacts	12
2.4 Les événements (cérémonies)	12
2.5 Découpage et estimation	13
2.6 Atelier 2 — Product Backlog & Sprint Planning	15
Quiz — Jour 2	15
Jour 3 — Kanban & gestion Agile avancée	16
3.1 Kanban : origines et principes	16
3.2 Visualisation, WIP, lead time, throughput	17
3.3 Optimisation des flux	18
3.4 Scrum vs Kanban vs ScrumBan	18
3.5 Atelier 3 — Tableau Kanban & simulation de flux	18
3.6 Les outils de gestion de projet agile	19
Évaluation finale du Module 1	20

Avant de commencer

Ce support alterne **théorie courte, exemples concrets et ateliers** à réaliser soi-même. Le fil rouge est une application mobile de **commande de repas**, « **QuickBite** », que l'on fait évoluer atelier après atelier — vous appliquez chaque notion immédiatement.

Comment lire ce support

Les encadrés bleus sont des **notes** (à mémoriser), les verts des **astuces**, les oranges des **points de vigilance** (erreurs fréquentes), les violets des **ateliers** (à faire vous-même).

Ce dont vous avez besoin

- Un compte gratuit sur **Jira** (ou **Trello** / **GitHub Projects**) pour les ateliers de board.
 - De quoi estimer en équipe : un jeu de **Planning Poker** (cartes physiques ou application en ligne).
 - En atelier de groupe : un tableau ou paperboard et des post-it de trois couleurs sont utiles, mais facultatifs.
-

Jour 1 — Fondamentaux de l'Agilité

🎯 Objectifs du Jour 1

- Expliquer d'où vient l'Agilité et le problème qu'elle résout.
- Citer et illustrer les 4 valeurs et les 12 principes du Manifeste.
- Distinguer une démarche prédictive d'une démarche adaptative.
- Définir MVP, valeur métier, boucle de feedback, DoR et DoD.
- Diagnostiquer un projet et proposer une trajectoire de transition.

1.1 Pourquoi l'Agilité ? Le contexte historique

Le problème de départ

Avant les années 2000, l'industrie logicielle pilotait presque tous ses projets en **cycle en cascade** (waterfall) : on spécifiait tout au début, on développait, puis on testait, puis on livrait — parfois 18 mois plus tard. Le constat, documenté par de nombreuses études (dont les rapports du *Standish Group / CHAOS*), était sévère :

- une part importante des projets dépassaient budget et délais ;
- beaucoup de fonctionnalités livrées **n'étaient jamais ou rarement utilisées** ;
- les besoins avaient changé entre la spécification et la livraison.

⚠️ Le piège du « tout prévoir »

Plus l'horizon de livraison est lointain, plus l'écart entre ce qui a été spécifié et ce dont l'utilisateur a réellement besoin se creuse. L'Agilité ne supprime pas l'incertitude : elle la rend gérable en raccourcissant les boucles.

La naissance du Manifeste (2001)

En février 2001, dix-sept praticiens (Kent Beck, Martin Fowler, Ken Schwaber, Jeff Sutherland, etc.) se réunissent à Snowbird (Utah). Ils venaient de méthodes différentes — eXtreme Programming, Scrum, DSDM, Crystal — mais partageaient un même rejet du formalisme lourd. Ils en sortent le **Manifeste pour le développement Agile de logiciels**.

L'Agilité n'est **pas** une méthode. C'est un **état d'esprit** (mindset) décrit par un manifeste, décliné ensuite en frameworks (Scrum, Kanban, XP, SAFe...) et en pratiques (TDD, pair programming, intégration continue...).

Repères chronologiques

Année	Jalon
1986	Takeuchi & Nonaka, « <i>The New New Product Development Game</i> » — métaphore du rugby (<i>scrum</i>)
1995	Première présentation formelle de Scrum (Schwaber & Sutherland)
1999	Kent Beck publie <i>Extreme Programming Explained</i>
2001	Manifeste Agile
2003	David J. Anderson applique Kanban au développement logiciel
2010+	Mise à l'échelle (SAFe, LeSS), DevOps, Agile « partout »

Idée reçue à corriger tout de suite

On confond souvent « Agile » avec « pas de documentation ni de plan ». C'est faux : Agile signifie **juste assez de documentation, au bon moment**, plutôt qu'un gros document figé écrit trop tôt. Pour ancrer la notion, repensez à un projet qui a mal tourné autour de vous : la plupart des causes (besoins mal compris, feedback trop tardif, périmètre figé) sont exactement ce que l'Agilité cherche à adresser.

1.2 Le Manifeste Agile : 4 valeurs, 12 principes

Les 4 valeurs

Le Manifeste s'énonce par quatre préférences. La formule est toujours « *A plutôt que B* » — **B garde de la valeur, mais A en a davantage.**

On valorise...	...plutôt que	Ce que ça veut dire concrètement
Les individus et leurs interactions	les processus et les outils	Un outil ne sauve pas une équipe qui ne se parle pas.
Des logiciels opérationnels	une documentation exhaustive	La mesure d'avancement = du logiciel qui marche, pas des pages de specs.
La collaboration avec le client	la négociation contractuelle	Le client est un partenaire continu, pas un signataire en fin de course.
L' adaptation au changement	le suivi d'un plan	Le plan sert à décider ; il évolue dès qu'on apprend du nouveau.

Erreur classique

« Agile, donc on ne documente pas / on ne planifie pas. » FAUX. Le Manifeste dit explicitement que les éléments de droite **ont de la valeur**. On déplace simplement le curseur.

Les 12 principes (regroupés pour l'enseignement)

Plutôt que de réciter les 12 principes, regroupez-les en 4 familles :

A. Satisfaire le client par la valeur livrée tôt et souvent 1. Priorité : satisfaire le client en livrant tôt et régulièrement de la valeur. 2. Accueillir le changement, même tardif, comme un avantage compétitif. 3. Livrer fréquemment (de quelques semaines à quelques mois, le plus court étant préféré).

B. Collaboration humaine au quotidien 4. Métier et développeurs travaillent ensemble quotidiennement. 5. Bâtir les projets autour de personnes motivées ; leur faire confiance. 6. Le dialogue en face à face est le moyen le plus efficace de communiquer.

C. La mesure, c'est le produit qui marche 7. Un logiciel opérationnel est la principale mesure d'avancement. 8. Rythme **soutenable** : tenir une cadence constante indéfiniment.

D. Excellence technique et amélioration continue 9. Une attention continue à l'excellence technique et à la conception. 10. La simplicité — maximiser la quantité de travail **non fait** — est essentielle. 11. Les meilleures architectures émergent d'équipes auto-organisées. 12. À intervalles réguliers, l'équipe réfléchit et **ajuste** son comportement.



Astuce d'animation

Distribuez les 12 principes imprimés et découpés. Chaque binôme range les bouts de papier dans les 4 familles ci-dessus, puis donne un exemple vécu pour 2 principes. Excellent pour ancrer sans réciter.

Mini-exercice (10 min)

Pour chaque situation, dites quelle valeur/principe est en jeu : 1. Le client demande un changement à mi-parcours. → *Valeur 4 / Principe 2*. 2. L'équipe décide elle-même comment se répartir le travail. → *Principe 11*. 3. On préfère une démo plutôt qu'un rapport d'avancement de 40 pages. → *Valeur 2 / Principe 7*.

1.3 Prédicatif (cycle en V) vs Adaptatif (itératif)

Le cycle en V / cascade

Besoins → Spécifications → Conception → Codage → Tests → Recette → Livraison
(tout est figé en amont, la valeur arrive à la toute fin)

- **Forces** : cadre rassurant, jalons contractuels nets, adapté quand le besoin est **stable et bien connu** (réglementaire, matériel, génie civil).
- **Faiblesses** : effet tunnel, feedback tardif, coût du changement très élevé en fin de cycle.

L'approche adaptative / itérative-incrémentielle

On livre par **incréments** : à chaque itération, un morceau de produit **utilisable** est produit, montré, et le plan est ajusté.

Critère	Prédictif (V)	Adaptatif (Agile)
Besoin	Stable, connu	Évolutif, incertain
Livraison de valeur	À la fin	À chaque itération
Gestion du changement	Coûteuse, encadrée	Attendue, intégrée
Mesure d'avancement	% de phases finies	Produit qui marche
Risque détecté	Tard	Tôt

■ Itératif ≠ incrémental

Incrémental : on ajoute des morceaux (on construit la maison pièce par pièce). **Itératif** : on repasse pour améliorer l'existant (on peint un premier jet, puis on raffine). Agile combine les deux : la fameuse illustration « skateboard → trottinette → vélo → voiture » plutôt que « roue → châssis → voiture ».

1.4 Concepts clés

MVP — Minimum Viable Product

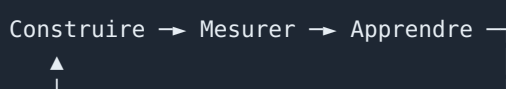
Le **produit minimum viable** est la plus petite version livrable qui permet d'**apprendre** auprès de vrais utilisateurs. Ce n'est pas un produit bâclé : c'est un produit **réduit mais cohérent**.

- *Mauvais MVP de QuickBite* : « on livre un demi-écran de paiement non fonctionnel ».
- *Bon MVP de QuickBite* : « commander **un** plat dans **un** restaurant et payer — bout en bout ». On apprend si les gens commandent.

Valeur métier

C'est le bénéfice apporté au client/utilisateur ou à l'entreprise : revenu, économie, réduction de risque, satisfaction. **On priorise par la valeur**, pas par la facilité technique.

Boucle de feedback (feedback loop)



Plus la boucle est courte, plus on corrige tôt et moins cher. C'est le cœur de l'Agilité.

Definition of Ready (DoR) & Definition of Done (DoD)

	Definition of Ready	Definition of Done
Quand ?	Avant d'entrer dans un sprint	Pour considérer un item terminé
Question	« Cette story est-elle prête à être développée ? »	« Ce travail est-il vraiment fini ? »
Exemples de critères	Story claire, estimée, critères d'acceptation définis, dépendances connues	Code revu, testé, intégré, documenté, déployable, démontré

⚠ Sans DoD, le « presque fini » s'accumule

Le syndrome « c'est fini à 90 % » détruit la prévisibilité. La DoD est une checklist partagée et **non négociable** par item.

1.5 Atelier 1 — Diagnostic et transition vers l'Agile

🔧 Atelier 1 — Durée 1 h 30 — Équipes de 4

Contexte : « FidExpress » est une PME qui livre un logiciel de fidélité en cascade. Cycle de 12 mois, beaucoup de retards, clients mécontents, specs obsolètes à la livraison.

Mission : diagnostiquer et proposer une trajectoire vers l'Agile.

Étapes : 1. (20 min) Lister les **symptômes** observés et les **causes racines** (utilisez les 5 pourquoi). 2. (25 min) Pour 3 causes, proposer une **pratique agile** qui y répond (livraison fréquente, démo, rétrospective...). 3. (25 min) Dessiner une **trajectoire de transition** en 3 étapes sur 6 mois (équipe pilote → généralisation). 4. (20 min) Restitution 5 min/équipe.

Livrable : une affiche « Symptôme → Cause → Pratique agile → Bénéfice attendu ».

Éléments de corrigé

Symptôme	Cause racine	Pratique agile	Bénéfice
Specs obsolètes à la livraison	Cycle trop long	Itérations 2-3 semaines + démo	Feedback tôt, specs vivantes
Effet tunnel	Pas de point produit	Sprint Review régulière	Visibilité client
Mêmes erreurs répétées	Pas de bilan	Rétrospective	Amélioration continue
Surcharge / burn-out	Plan irréaliste	Rythme soutenable + WIP	Cadence tenable

💡 Le piège à éviter dans cet atelier

Ne proposez jamais « faire du Scrum » sans relier ce choix à un **symptôme** précis : on part toujours du **problème**, pas de l'outil. Gardez en tête la conclusion : « Agile n'est pas un objectif, c'est un **moyen** de livrer de la valeur plus vite et plus sûrement. »

Quiz — Jour 1

1. Vrai/Faux : « Être agile signifie ne pas planifier. » → **Faux.**
 2. Citez deux des quatre valeurs du Manifeste.
 3. Quelle est la différence entre DoR et DoD ?
 4. Qu'est-ce qu'un MVP, en une phrase ?
 5. Donnez un cas où le cycle en V reste pertinent. → *(Besoin stable/réglementaire, ex. logiciel embarqué certifié.)*
 6. Que mesure-t-on en priorité pour suivre l'avancement en Agile ? → *Du logiciel opérationnel.*
-

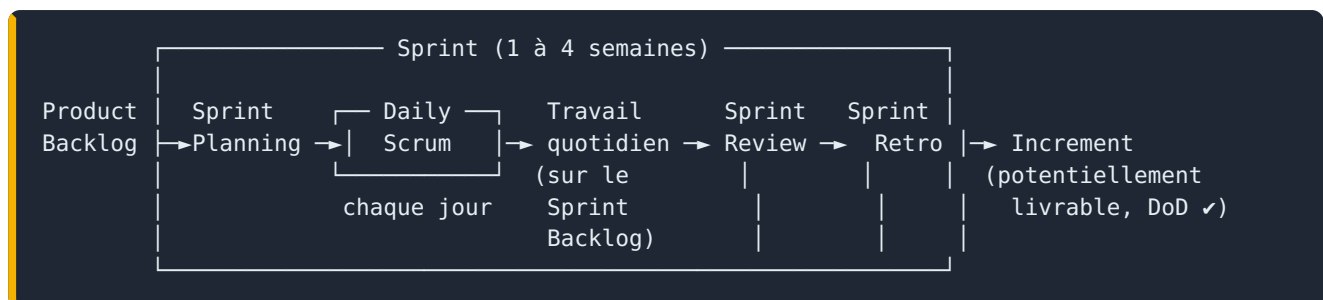
Jour 2 — Scrum : cadre, rôles et cérémonies

🎯 Objectifs du Jour 2

- Décrire le cadre Scrum : rôles, artefacts, événements, et leur articulation.
- Distinguer clairement les responsabilités du PO, du SM et des Développeurs.
- Animer chacune des cinq cérémonies et en connaître la timebox.
- Découper un besoin en Epics → User Stories → Tasks et estimer en points.
- Construire un Product Backlog priorisé et planifier un premier sprint.

2.1 Le cadre Scrum en un coup d'œil

Scrum est un **cadre** (framework) léger pour résoudre des problèmes complexes en livrant des produits de la plus haute valeur possible. Il repose sur l'**empirisme** : transparence, inspection, adaptation.



Les 3 piliers empiriques :

- **Transparence** — tout le monde voit le même état réel (backlog, board, incréments).
- **Inspection** — on examine régulièrement le produit et la façon de travailler.
- **Adaptation** — on ajuste dès qu'un écart est constaté.

Les 5 valeurs Scrum : Engagement, Focus, Ouverture, Respect, Courage.

2.2 Les rôles (« responsabilités »)

Scrum définit **une seule équipe** (Scrum Team) sans hiérarchie interne, regroupant trois responsabilités.

Product Owner (PO) — « Quoi et pourquoi »

Le PO **maximise la valeur** du produit. Il est le **garant unique** du Product Backlog.

Responsabilités : - Définir et porter la **vision produit**. - **Constituer, ordonner et clarifier** le Product Backlog. - **Prioriser** par la valeur (et le risque, le coût, les dépendances). - Être disponible pour l'équipe (répondre aux questions, valider). - Décider de ce qui entre dans un incrément et accepter/refuser le résultat.

Outils du PO : **story mapping**, techniques de priorisation (**MoSCoW**, **WSJF**, **valeur/effort**), critères d'acceptation, roadmap.

Le PO décide, il ne code pas pour l'équipe

Le PO arbitre la valeur ; il **n'impose pas** les solutions techniques. Un bon PO dit « j'ai besoin que l'utilisateur puisse payer en un clic », pas « ajoute un bouton bleu à tel endroit ».

Scrum Master (SM) — « Comment bien travailler »

Le SM est un **leader serviteur** (servant leader) : il est responsable de l'efficacité de l'équipe Scrum et de la bonne application de Scrum.

Responsabilités : - **Faciliter** les événements et fluidifier la collaboration. - **Lever les obstacles** (impediments) qui ralentissent l'équipe. - **Coacher** l'équipe vers l'auto-organisation et l'amélioration continue. - **Protéger** l'équipe des interruptions et du sur-engagement. - Aider l'organisation à adopter Scrum.

Le SM n'est pas un chef de projet

Il ne distribue pas les tâches, ne fixe pas les délais à la place de l'équipe, n'évalue pas individuellement les personnes. Son pouvoir est l'influence, pas l'autorité.

Les Développeurs — « Comment réaliser »

Toute personne qui contribue à l'incrément (dev, test, design, ops...). Ils sont **auto-organisés** et **pluridisciplinaires**.

Responsabilités : - Créer un plan de sprint (Sprint Backlog). - Respecter la **Definition of Done**. - S'adapter chaque jour vers l'objectif de sprint. - Se tenir mutuellement responsables en tant que professionnels.

	Product Owner	Scrum Master	Développeurs
Focus	La valeur	Le fonctionnement	La réalisation
Possède	Le Product Backlog	Le processus Scrum	Le Sprint Backlog
Question type	« Quoi, dans quel ordre ? »	« Qu'est-ce qui nous bloque ? »	« Comment, et combien ? »

2.3 Les artefacts

Artefact	Description	Engagement associé
Product Backlog	Liste ordonnée et vivante de tout ce qui pourrait être nécessaire au produit	Product Goal (objectif produit)
Sprint Backlog	Sélection d'items pour le sprint + plan pour les réaliser	Sprint Goal (objectif de sprint)
Increment	Somme des items « Done » ; doit être utilisable	Definition of Done

Un artefact = un engagement

Scrum 2020 associe à chaque artefact un « engagement » qui donne la direction et permet de mesurer le progrès : Product Goal, Sprint Goal, Definition of Done.

2.4 Les événements (cérémonies)

Le **Sprint** est le conteneur de tous les autres événements : durée fixe de 1 à 4 semaines, sans interruption, enchaîné immédiatement par le suivant.

Événement	Timebox (sprint 2 sem.)	Qui	But
Sprint Planning	≤ 4 h	Toute l'équipe	Définir le <i>Sprint Goal</i> et le plan
Daily Scrum	15 min/jour	Développeurs	Se synchroniser, ajuster le plan du jour
Sprint Review	≤ 2 h	Équipe + parties prenantes	Inspecter l'incrément, recueillir le feedback
Sprint Retrospective	≤ 1 h 30	Toute l'équipe	Améliorer la façon de travailler

Sprint Planning — répond à 3 questions

1. **Pourquoi** ce sprint a-t-il de la valeur ? → Sprint Goal.
2. **Quoi** peut-on livrer ? → items sélectionnés.
3. **Comment** le réaliser ? → plan / tâches.

Daily Scrum — format classique

Chaque développeur : *Qu'ai-je fait hier qui aide l'objectif ? Que vais-je faire aujourd'hui ? Qu'est-ce qui me bloque ?* Ce n'est **pas** un reporting au SM, mais une synchro entre devs.

 Garder le Daily à 15 min

Debout, à heure fixe, devant le board. Les sujets longs se traitent juste après, entre les seuls concernés (« after-party »).

Sprint Review vs Rétrospective

- **Review** = on inspecte **le PRODUIT** avec les parties prenantes.
- **Rétro** = on inspecte **le PROCESSUS** entre membres de l'équipe.

Formats de rétrospective utiles

- **Start / Stop / Continue.**
- **Mad / Sad / Glad.**
- **4 L** : Liked, Learned, Lacked, Longed for.

2.5 Découpage et estimation

Hiérarchie

```
Epic (gros besoin)
├── User Story (besoin utilisateur livrable en 1 sprint)
│   ├── Task (travail technique de quelques heures)
```

La User Story

Format canonique :

En tant que [rôle], **je veux** [action] **afin de** [bénéfice].

Exemple QuickBite :

En tant que **client**, je veux **payer ma commande par carte** afin de **finaliser mon achat sans quitter l'appli**.

Les critères INVEST (bonne story)

Lettre	Signifie	Question
I	Independent	Peut-on la faire seule ?
N	Negotiable	Le « comment » reste-t-il ouvert ?
V	Valuable	Apporte-t-elle de la valeur ?
E	Estimable	Peut-on l'estimer ?
S	Small	Tient-elle dans un sprint ?
T	Testable	A-t-elle des critères d'acceptation ?

Critères d'acceptation (format Gherkin)

```

Étant donné que je suis un client avec un panier non vide
Quand je saisis une carte valide et que je confirme
Alors le paiement est accepté
Et je reçois un récapitulatif de commande
  
```

Estimation relative — Planning Poker

On estime en **points de story** (suite de Fibonacci : 1, 2, 3, 5, 8, 13, 20...), une mesure **relative** de la complexité/effort, pas des heures.

1. Le PO présente la story.
2. Chacun choisit une carte en secret, puis on révèle simultanément.
3. Les extrêmes s'expliquent, on rejoue jusqu'au consensus.

Pourquoi pas des heures ?

Les estimations en heures donnent une fausse précision et sont vite fausses. Les points capturent la **taille relative** ; la **vélocité** (points livrés/sprint) sert ensuite à prévoir. La référence est « la plus petite story = 1, 2 ou 3 ».

MoSCoW pour prioriser

- **Must** — indispensable au MVP.
- **Should** — important mais contournable.
- **Could** — souhaitable si temps.
- **Won't (now)** — hors périmètre actuel.

2.6 Atelier 2 — Product Backlog & Sprint Planning

Atelier 2 — Durée 2 h — Équipes de 4-5

Produit : QuickBite (commande de repas). Vision : « permettre à un client de commander un repas dans un restaurant proche et de payer en quelques clics ».

Mission : construire le Product Backlog, l'ordonner, estimer, puis planifier le Sprint 1.

Étapes : 1. (25 min) **Brainstorm** : produire au moins 12 idées d'items (post-it). 2. (25 min) Écrire **6 User Stories** au format canonique + critères d'acceptation (Gherkin) pour 2 d'entre elles. 3. (20 min) **Estimer** en Planning Poker (Fibonacci). 4. (20 min) **Prioriser** en MoSCoW ; définir le **Product Goal**. 5. (20 min) **Sprint Planning** : capacité de l'équipe = 20 points ; choisir les items du Sprint 1 et formuler un **Sprint Goal**. 6. (10 min) Restitution.

Exemple de corrigé (extrait)

#	User Story	Estimation	MoSCoW
1	Voir la liste des restaurants proches	5	Must
2	Consulter le menu d'un restaurant	3	Must
3	Ajouter un plat au panier	3	Must
4	Payer par carte	8	Must
5	Suivre l'état de ma commande	5	Should
6	Noter le restaurant	2	Could

Sprint Goal possible : « Un client peut commander un plat dans un restaurant et payer — parcours bout en bout. » → Stories 1, 2, 3, 4 (= 19 points, dans la capacité).

Erreur fréquente dans cet atelier

Vérifiez que vos stories respectent **INVEST**. L'erreur classique : écrire des « stories » purement techniques (« créer la table SQL ») — ce sont des **tâches**, pas des stories porteuses de valeur pour l'utilisateur. Et rappelez-vous : le **Sprint Goal** donne la cohérence d'ensemble, ce n'est pas une simple addition d'items.

Quiz — Jour 2

1. Quelle est la timebox du Daily Scrum ? → **15 min.**
2. Qui est responsable du Product Backlog ? → **Le Product Owner.**
3. Le Scrum Master attribue-t-il les tâches ? → **Non.**
4. Différence entre Sprint Review et Rétrospective ?
5. Que signifie le « V » d'INVEST ?
6. Pourquoi estimer en points plutôt qu'en heures ?

Jour 3 — Kanban & gestion Agile avancée

🎯 Objectifs du Jour 3

- Énoncer les principes et pratiques de Kanban.
- Mettre en place un tableau, limiter le WIP et lire les métriques de flux.
- Choisir entre Scrum, Kanban et ScrumBan selon le contexte.
- Construire et faire vivre un board dans un outil moderne.

3.1 Kanban : origines et principes

Kanban (« panneau » / « carte visuelle » en japonais) vient du **Toyota Production System** (juste-à-temps). Appliqué au logiciel par David J. Anderson (2010), c'est une **méthode d'amélioration évolutive** : on **part de l'existant** et on optimise le flux, sans imposer de rôles ni d'itérations.

Les 4 principes de conduite du changement

1. Commencer par ce que vous faites **maintenant**.
2. Accepter de poursuivre par un changement **évolutif et incrémental**.
3. Respecter le processus, les rôles et responsabilités **actuels**.
4. Encourager le **leadership à tous les niveaux**.

Les 6 pratiques fondamentales

1. **Visualiser** le travail (board).
2. **Limiter le travail en cours** (WIP).
3. **Gérer le flux** (le rendre fluide et prévisible).
4. **Rendre les règles explicites** (politiques de colonne, DoD).
5. **Mettre en place des boucles de feedback** (cadences de revue).
6. **S'améliorer collaborativement** (modèles, expérimentation).

3.2 Visualisation, WIP, lead time, throughput

Un tableau Kanban simple

À FAIRE	EN COURS (3)	REVUE (2)	TERMINÉ
[C7] [C8] [C9]	[C3] [C4] [C5]	[C2]	[C1]

▲ limite WIP=3 ▲ limite WIP=2

Limiter le WIP (Work In Progress)

La règle centrale de Kanban : **on borne le nombre de cartes par colonne**. Conséquences vertueuses : - on **finir** avant de commencer du nouveau (« stop starting, start finishing ») ; - les **goulots d'étranglement** deviennent visibles ; - le **lead time** baisse (loi de Little).

! Le multitâche tue le débit

Sans limite de WIP, tout le monde commence tout, rien ne se termine, le lead time explose. Limiter le WIP est **contre-intuitif mais le levier n°1** de Kanban.

Les métriques de flux

Métrique	Définition	Sert à
Lead time	Temps entre la demande et la livraison (vu du client)	Prévoir les délais
Cycle time	Temps entre le début du travail et la fin	Améliorer l'exécution
Throughput (débit)	Nombre d'items terminés / unité de temps	Mesurer la capacité réelle
WIP	Nombre d'items en cours	Réguler le flux

Loi de Little : $\text{Lead time} \approx \text{WIP} / \text{Throughput}$. Diminuer le WIP (à débit constant) réduit mécaniquement le lead time.

Diagramme de flux cumulé (CFD)

Graphe empilé du nombre d'items par état dans le temps. Une bande qui **s'élargit** = accumulation (goulot) ; des bandes **parallèles et fines** = flux sain.

3.3 Optimisation des flux

- **Classes de service** : Standard, Urgent (expedite), Date fixe, Intangible — avec des règles distinctes.
- **Limiter pour révéler** : baisser une limite WIP fait apparaître les blocages.
- **Cadences** : réunion de réapprovisionnement (replenishment), revue de livraison, revue de risques.
- **Mesurer puis agir** : on n'optimise que ce qu'on observe (CFD, run charts).

3.4 Scrum vs Kanban vs ScrumBan

Critère	Scrum	Kanban
Cadence	Itérations fixes (sprints)	Flux continu
Rôles	PO, SM, Devs	Aucun imposé
Engagement	Sprint Goal	Pas d'engagement par lot
Limite de travail	Capacité du sprint	Limite WIP par colonne
Changement en cours	Évité pendant le sprint	Possible à tout moment
Idéal pour	Développement produit itératif	Flux continu, support, ops

ScrumBan = hybride : on garde des éléments de Scrum (rôles, certaines cérémonies, rétro) mais on pilote par le **flux et des limites de WIP** plutôt que par des sprints rigides. Utile pour les équipes de **maintenance/support** ou en transition.

💡 Quand choisir quoi ?

Scrum : nouveau produit, besoin de cadence et d'objectifs.

Kanban : flux entrant imprévisible (support, infogérance, contenu).

ScrumBan : équipe Scrum noyée par l'urgent, ou transition douce.

3.5 Atelier 3 — Tableau Kanban & simulation de flux

🔧 Atelier 3 — Durée 1 h 30 — Équipes de 4-5

Mission : modéliser le flux de support de QuickBite, poser des limites de WIP, puis simuler 5 « jours » pour observer un goulot et l'améliorer.

Étapes : 1. (15 min) Définir les colonnes (ex. *Backlog* → *Analyse* → *Dev* → *Test* → *Déployé*) et les **règles** de chaque colonne. 2. (15 min) Poser des **limites de WIP** réalistes. 3. (30 min) **Simulation** : tirer 2 cartes/jour en entrée, avancer les cartes selon une capacité par colonne (fixez une cadence simple, ou lancez un dé pour simuler la variabilité). Noter le lead time de chaque carte. 4. (15 min) Identifier le **goulot**, baisser/déplacer une limite, rejouer 2 jours. 5. (15 min) Restitution : qu'est-ce qui a changé sur le lead time / le throughput ?

Éléments de corrigé

- Le goulot apparaît souvent en **Test** (ou Revue) : les cartes s'y empilent.
- Réduire le WIP amont ou renforcer la colonne goulot (entraide, swarming) **lisse le flux**.
- Message clé : **optimiser le tout, pas chaque poste isolément** (théorie des contraintes).

3.6 Les outils de gestion de projet agile

Un cadre comme Scrum a besoin d'un **outil** pour vivre au quotidien : ordonner le backlog, tenir le board, planifier les sprints, suivre l'avancement. Voici les principaux acteurs du marché.

Outil	Éditeur / type	Points forts	Limites	Idéal pour
Jira	Atlassian	Le plus complet pour Scrum et Kanban : backlog, sprints, burndown, story points, workflows sur mesure, reporting, agile à l'échelle (SAFe)	Riche donc complexe ; lourd pour une petite équipe	Équipes produit, de la PME au grand groupe
Azure DevOps Boards	Microsoft	Boards + Repos + Pipelines dans une même suite	Surtout pertinent dans l'écosystème Azure	Équipes déjà chez Microsoft
GitHub Projects	GitHub	Collé au code, aux issues et aux PR	Gestion de projet plus basique	Équipes de développeurs
GitLab	GitLab	Issues, boards et CI/CD réunis	Moins riche que Jira sur l'agile pur	Équipes DevOps intégrées
Trello	Atlassian	Ultra simple, cartes Kanban	Ni sprints ni estimation en natif	Petites équipes, Kanban léger
ClickUp / Asana / Monday	éditeurs divers	Polyvalents (tâches, docs, vues)	Pas spécialisés Scrum	Coordination transverse
Linear	Linear	Rapide, épuré, orienté produit	Reporting entreprise limité	Startups, équipes produit

Notre choix : Jira — et pourquoi

Pour le fil rouge QuickBite, nous retenons **Jira**. Ce n'est pas le plus simple des outils, mais c'est celui qui **colle le mieux à un vrai projet Scrum** et au monde professionnel. La justification, **concept par concept** :

- **Il parle nativement Scrum** : **backlog** ordonnancable, **sprints** qu'on planifie, démarre et clôture, **board Scrum** et **board Kanban** — exactement les artefacts et événements du cadre, sans bricolage.

- **Il modélise la hiérarchie Epic → User Story → Task**, gère les **story points** et l'estimation (Planning Poker via extension).
- **Il calcule les métriques agiles** sans effort : **burndown / burnup, vélocité, diagramme de flux cumulé (CFD)**, temps de cycle — le PO et le Scrum Master pilotent avec des **données réelles**.
- **Ses workflows sont personnalisables** : on y inscrit la **Definition of Done** et les règles de transition.
- **Il passe à l'échelle** (d'une équipe à plusieurs équipes coordonnées, SAFe), ce qui en fait le **standard des grandes organisations**.
- **Il s'intègre à la chaîne DevOps** (Module 2) : liaison avec **Bitbucket / GitHub / GitLab**, visibilité de la **CI/CD**, traçabilité **commit ↔ ticket**.
- **Employabilité** : Jira est, de loin, l'outil le plus répandu en entreprise — une compétence directement valorisable.

⚠ Jira n'est pas toujours le bon choix

Sa richesse a un prix : **courbe d'apprentissage**, configuration parfois lourde, tarif qui grimpe au-delà de quelques utilisateurs. Pour un projet personnel ou une équipe de deux ou trois personnes, **Trello** ou **GitHub Projects** suffisent. On choisit l'outil **adapté au besoin**, pas le plus gros par principe.

🔧 Mini-atelier outil — 30 min

Sur **Jira** (offre Cloud gratuite jusqu'à 10 utilisateurs) : créez un projet **Scrum** « QuickBite », alimentez le **backlog** avec les 6 User Stories de l'Atelier 2, attribuez **story points** et étiquettes **MoSCoW**, planifiez et **démarrez un sprint**, déplacez 2 tickets sur le board, puis ouvrez le **rapport burndown**. *Variante légère* : refaites-le sur **Trello** ou **GitHub Projects** et comparez ce qu'on gagne (simplicité) et ce qu'on perd (sprints, estimation, métriques).

Évaluation finale du Module 1

Partie A — QCM (10 questions, 1 pt)

1. Le Manifeste Agile date de : a) 1995 b) **2001** c) 2010.
2. Qui priorise le Product Backlog ? **PO**.
3. Timebox du Sprint Planning (sprint 2 sem.) : **≤ 4 h**.
4. Le WIP, c'est : **le travail en cours**.
5. Loi de Little : **Lead time ≈ WIP / Throughput**.
6. INVEST : le « S » = **Small**.
7. Une rétrospective inspecte : **le processus**.
8. ScrumBan combine : **flux Kanban + éléments Scrum**.
9. La DoD sert à : **définir ce qui est « terminé »**.
10. Estimer en points plutôt qu'en heures car : **taille relative, plus fiable**.

Partie B — Étude de cas (10 pts)

On vous confie une équipe de **support applicatif** noyée sous des tickets urgents et imprévisibles, mais qui doit aussi avancer sur des évolutions planifiées.

1. Quel cadre recommandez-vous (Scrum, Kanban, ScrumBan) et **pourquoi** ? (4 pts)
2. Proposez un board avec colonnes, **limites de WIP** et une **classe de service « urgent »**. (3 pts)
3. Quelles **2 métriques** suivez-vous pour prouver l'amélioration ? (3 pts)

Corrigé synthétique Partie B

1. **ScrumBan/Kanban** : flux entrant imprévisible + besoin de visualiser et limiter le WIP ; les sprints rigides cassent sur l'urgence.
2. *Backlog* → *Analyse(3)* → *En cours(3)* → *Test(2)* → *Fait* + couloir « Expedite » à WIP=1 prioritaire.
3. **Lead time** (délai vu du client) et **throughput** (débit), suivis via un **CFD**.

✓ Clôture du Module 1

Faites une rétrospective « à chaud » de la formation (Start/Stop/Continue) : vous illustrez la pratique tout en récoltant du feedback. Distribuez la grille d'auto-évaluation des compétences (Agile, Scrum, Kanban : 1 à 5).